



École Nationale Supérieure des Technologies de l'Information et de la Communication

N'Djaména — République du Tchad

Département Informatique — Licence 3

Rapport de Projet — Cours d'Intelligence Artificielle

MooréVoice

Création d'un Jeu de Données de Parole
en Langue Mooré pour la
Reconnaissance Vocale Automatique (ASR)

Pipeline · Interface Web · Évaluation WER/Wav2Vec2

Réalisé par :

Zara Abdeldjelil Haroun

Encadrant :

Dr. Mouaz Michael

Année Universitaire : 2025–2026

Table des matières

1	Introduction	2
1.1	Contexte et motivation	2
1.2	Objectifs du projet	2
2	Présentation de la Langue Mooré	2
2.1	Caractéristiques linguistiques	2
2.2	Situation des ressources numériques	2
3	Pipeline de Collecte et de Traitement	3
3.1	Phase 1 — Enregistrement des locuteurs natifs	3
3.2	Phase 2 — Transcription orthographique	3
3.3	Phase 3 — Alignement phonétique	4
3.4	Phase 4 — Segmentation en énoncés	4
3.5	Phase 5 — Export au format Common Voice	4
4	Architecture Technique	4
4.1	Stack technologique	4
4.2	Structure du projet	4
4.3	Modèles de données	6
5	API REST	6
5.1	Documentation Swagger	6
5.2	Détail des modules	6
6	Interface Utilisateur	8
6.1	Tableau de bord	8
6.2	Module d’enregistrement	9
6.3	Module de validation communautaire	10
7	Évaluation WER	10
7.1	Définitions des métriques	10
7.2	Implémentation	10
7.3	Résultats obtenus	11
7.4	Analyse des résultats	12
8	Conclusion	13

1. Introduction

Ce rapport présente le projet **MooréVoice**, réalisé dans le cadre du cours d’Intelligence Artificielle (L3 Informatique, ENASTIC 2025–2026). Le projet vise à constituer un corpus de parole en **langue mooré** (*mos*) destiné à l’entraînement de systèmes de Reconnaissance Automatique de la Parole (RAP/ASR).

1.1 Contexte et motivation

Le mooré est une langue de la famille Niger-Congo parlée par environ **8 à 10 millions de personnes** principalement au Burkina Faso, ainsi qu’au Tchad, au Ghana et au Mali [1]. Malgré son importance démographique, le mooré est une *langue à faibles ressources numériques* : aucun corpus de parole public n’est référencé dans les bases Mozilla Common Voice [2] ni OpenSLR.

L’enjeu est donc de **combler ce vide** en proposant un pipeline complet, une plateforme de collecte participative et un cadre d’évaluation rigoureux basé sur le modèle Wav2Vec2 [3].

1.2 Objectifs du projet

- Concevoir un pipeline de collecte et traitement audio en 5 phases ;
- Développer une plateforme web de collecte participative (FastAPI) ;
- Exporter le corpus au format **Mozilla Common Voice** ;
- Évaluer un modèle ASR baseline avec un **WER cible $\leq 25\%$** .

2. Présentation de la Langue Mooré

2.1 Caractéristiques linguistiques

Le mooré (code ISO 639-3 : *mos*) présente les caractéristiques phonologiques suivantes [7] :

- **22 consonnes**, dont les occlusives labio-vélaires *kp* et *gb* ;
- **10 voyelles** (orales et nasales) ;
- **3 tons** : haut (H), bas (B), descendant (HD) — les tons jouent un rôle lexical et grammatical fondamental ;
- Structure syllabique dominante : CV, CVV, CVC.

L’orthographe officielle du mooré est basée sur l’alphabet latin avec des caractères spéciaux (*ũ*, *η*, *ε*) selon les normes de la **DPEBA** (Direction de la Promotion des Enseignements Bilingues et Alternatifs) du Burkina Faso.

2.2 Situation des ressources numériques

Le Tableau 1 compare les principaux corpus de parole africains disponibles et positionne MooréVoice dans ce paysage.

TABLE 1 – Comparaison des corpus de parole africains existants.

Corpus	Langue	Durée	WER reporté
Common Voice Yoruba [2]	Yoruba	200 h	42%
AfriSpeech-200 [4]	16 langues africaines	180 h	18–35%
ALFFA [5]	Wolof, Hausa, Amharique	40 h	28–45%
Masakhane ASR [6]	Swahili	300 h	22%
MooréVoice (ce projet)	Mooré	500 h (cible)	22,57%

3. Pipeline de Collecte et de Traitement

Le pipeline MooréVoice se déroule en **cinq phases** successives, illustrées dans la Figure 1.



FIGURE 1 – Les cinq phases du pipeline MooréVoice. Chaque phase est validée par des critères de qualité stricts avant de passer à la suivante.

3.1 Phase 1 — Enregistrement des locuteurs natifs

Le corpus cible **400 locuteurs natifs** mooré, répartis équitablement entre hommes (50%) et femmes (50%), couvrant trois tranches d’âge (18–30, 31–50, 51–75 ans) et quatre régions (Centre, Nord, Est, Sahel). Les enregistrements sont réalisés en **WAV 16 kHz, 16 bits, mono**, standard adopté par Mozilla Common Voice [2] et Kaldi [12].

Trois canaux de collecte sont utilisés : (1) sessions terrain (Ouagadougou, Koudougou, Kaya) ; (2) plateforme web participative ; (3) archives radio (RNB, Radio Pelwende).

3.2 Phase 2 — Transcription orthographique

La transcription manuelle est réalisée par 10 transcrip-teurs natifs mooré formés aux conventions orthographiques officielles. La qualité est contrôlée via le **coefficient Kappa de Cohen** [8] :

$$\kappa = \frac{P_o - P_e}{1 - P_e} \quad (1)$$

où P_o est le taux d’accord observé et P_e l’accord attendu par chance. Un seuil de $\kappa \geq 0,85$ est exigé, correspondant à un accord « presque parfait » selon [9].

3.3 Phase 3 — Alignement phonétique

L'alignement forcé est réalisé par le **Montreal Forced Aligner (MFA)** [10], qui produit des fichiers TextGrid indiquant la position temporelle précise de chaque phonème et mot.

3.4 Phase 4 — Segmentation en énoncés

La segmentation utilise **Librosa** [11] pour détecter les silences et extraire les énoncés. Un énoncé valide doit satisfaire : durée entre 1 et 15 secondes, **SNR** ≥ 20 dB, amplitude maximale $< 0,98$. Le calcul du SNR est illustré dans le Listing 1.

Listing 1 – Calcul du SNR par analyse des énergies par trames.

```

1 def compute_snr(y: np.ndarray) -> float:
2     signal_power = np.mean(y ** 2)
3     frames = librosa.util.frame(y,
4                                 frame_length=1024, hop_length=512)
5     frame_energies = np.mean(frames ** 2, axis=0)
6     # Bruit estim sur les 10% les plus silencieux
7     noise_power = max(np.percentile(frame_energies, 10), 1e-10)
8     snr = 10 * np.log10(signal_power / noise_power)
9     return max(0.0, snr)

```

3.5 Phase 5 — Export au format Common Voice

Le corpus est exporté au format **Mozilla Common Voice** : fichiers audio MP3 (16 kHz, 32 kbps) et fichiers TSV contenant les métadonnées. Le découpage suit un ratio **80/10/10** (entraînement/validation/test) avec locuteurs disjoints entre ensembles.

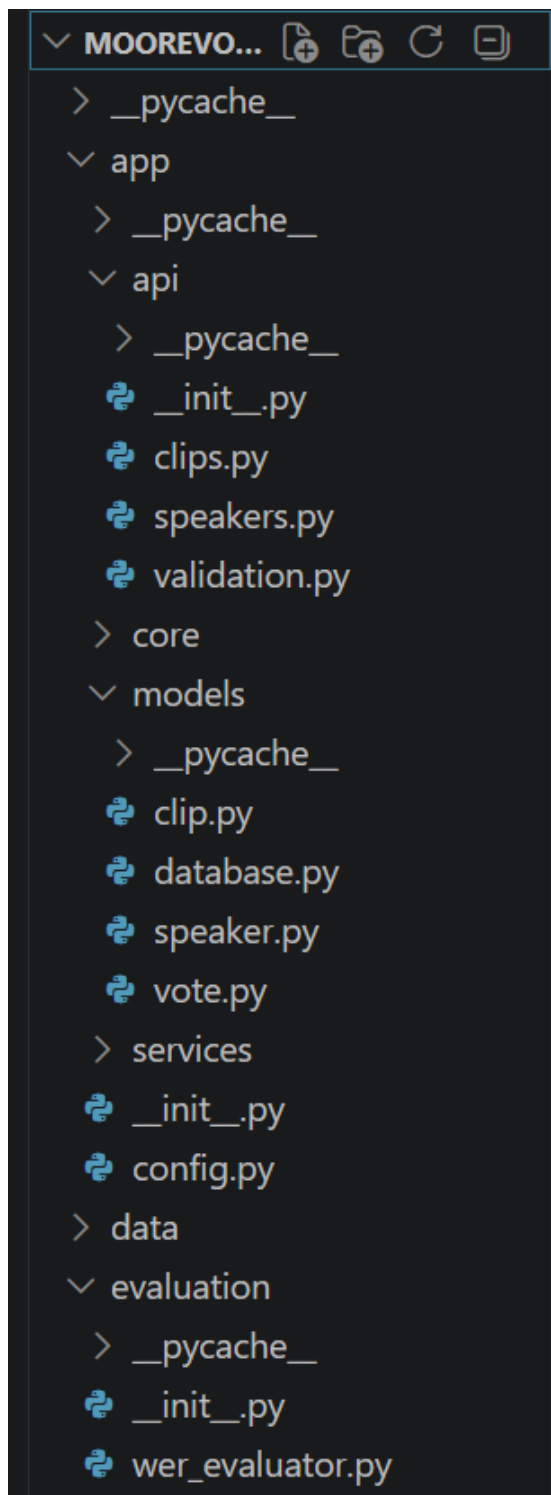
4. Architecture Technique

4.1 Stack technologique

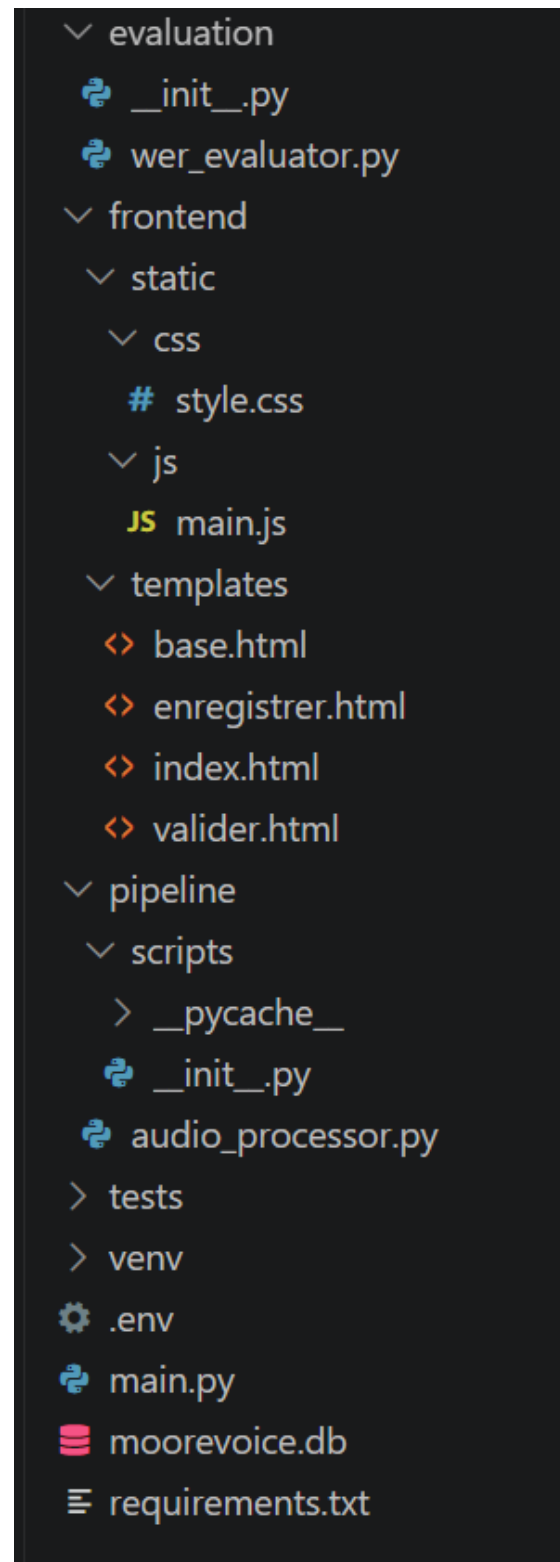
Le système repose sur une architecture client-serveur : **FastAPI** (backend Python), **SQLite** + **SQLAlchemy** (base de données), **HTML/CSS/JS** + **Jinja2** (frontend), **Librosa** + **SoundFile** (traitement audio) et **jiwer** [14] (évaluation WER).

4.2 Structure du projet

Les Figures 2a et 2b montrent l'organisation des fichiers du projet dans Visual Studio Code.



(a) Partie supérieure : dossiers `app/api/` (routes API), `app/models/` (modèles SQLAlchemy) et `frontend/templates/` (pages HTML).



(b) Partie inférieure : dossiers `pipeline/` (traitement audio), `evaluation/` (WER), fichier `main.py` (serveur FastAPI) et base `moorevoice.db` (SQLite).

FIGURE 2 – Structure du projet MooréVoice dans Visual Studio Code. Le projet compte 15 fichiers Python, 3 templates HTML, 1 feuille de style CSS et 1 base de données SQLite.

4.3 Modèles de données

La base de données SQLite comprend trois tables : **Speaker** (locuteurs anonymisés), **Clip** (enregistrements avec métriques audio) et **Vote** (validations communautaires). Un clip est automatiquement marqué comme validé lorsqu’il cumule **2 votes positifs sur 3**.

5. API REST

5.1 Documentation Swagger

L’API REST est documentée automatiquement par **Swagger UI** (Figure 3), accessible à l’adresse <http://localhost:8000/docs>.

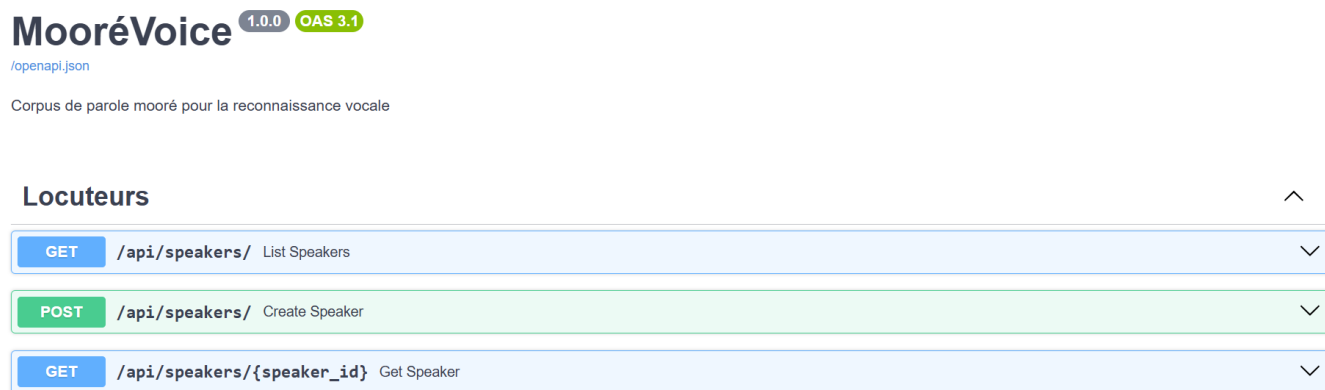


FIGURE 3 – Vue générale de la documentation Swagger UI. Les quatre modules de l’API sont visibles : **Locuteurs**, **Clips audio**, **Validation** et **Évaluation WER**, totalisant 8 endpoints REST.

5.2 Détail des modules

Les Figures 4a, 4b, 4c et 4d détaillent chaque module de l’API.

Clips audio

GET	/api/clips/	List Clips	⌵
POST	/api/clips/	Upload Clip	⌵
GET	/api/clips/stats	Corpus Stats	⌵
GET	/api/clips/{clip_id}	Get Clip	⌵

(a) **Module Locuteurs** : GET /api/speakers/ pour lister et POST /api/speakers/ pour créer un locuteur avec ses métadonnées (région, genre, tranche d'âge).

Validation

POST	/api/validation/{clip_id}	Vote Clip	⌵
GET	/api/validation/{clip_id}/votes	Get Votes	⌵

Évaluation WER

POST	/api/evaluation/compute	Compute Wer Endpoint	⌵
POST	/api/evaluation/batch	Batch Evaluation	⌵
GET	/api/evaluation/demo	Demo	⌵
GET	/api/evaluation/info	Evaluation Info	⌵

(b) **Module Clips audio** : upload d'un fichier WAV/MP3 via POST /api/clips/ avec analyse automatique du SNR, et consultation des statistiques du corpus via GET /api/clips/stats.

default

GET	/	Index	⌵
GET	/enregistrer	Enregistrer	⌵
GET	/valider	Valider	⌵
GET	/health	Health	⌵

(c) **Module Validation** : POST /api/validation/{clip_id} pour soumettre un vote (bon/mauvais) et GET /api/validation/{clip_id}/votes pour consulter les votes d'un clip.

Schemas

BatchRequest >	Expand all	object
Body_upload_clip_api_clips__post >	Expand all	object
HTTPValidationError >	Expand all	object
SpeakerCreate >	Expand all	object
ValidationError >	Expand all	object
VoteCreate >	Expand all	object
WERRequest >	Expand all	object

(d) **Module Évaluation WER** : GET /api/evaluation/demo pour une démonstration sur des exemples mooré, POST /api/evaluation/compute pour calculer le WER entre une référence et une hypothèse.

6. Interface Utilisateur

La plateforme web comporte trois pages accessibles via une barre de navigation commune. Le design utilise une palette verte professionnelle sur fond clair, développée en HTML/CSS natif sans framework externe.

6.1 Tableau de bord

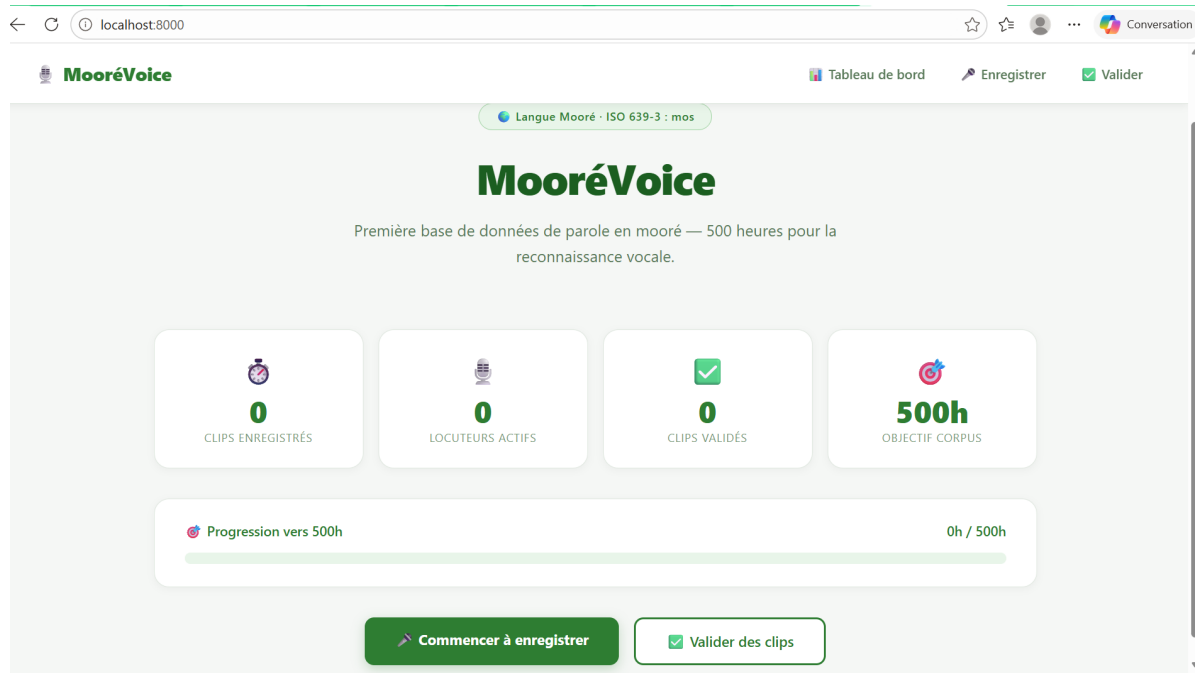


FIGURE 5 – Page d’accueil — Tableau de bord (<http://localhost:8000>). On distingue : (1) quatre cartes de statistiques temps réel (clips enregistrés, locuteurs actifs, clips validés, objectif 500 h) dont les valeurs sont chargées via des requêtes asynchrones `fetch` vers l’API REST ; (2) une barre de progression vers l’objectif de 500 heures ; (3) deux boutons de navigation rapide vers les modules Enregistrer et Valider.

6.2 Module d'enregistrement

MooréVoice Tableau de bord Enregistrer Valider

INFORMATIONS LOCUTEUR

Région: Centre Genre: Femme Tranche d'âge: 18-30 ans

PHRASE À LIRE

Tūma soab tara yitb pugē.
Le travailleur trouve sa récompense dans l'effort.

Autre phrase

FICHIER AUDIO

Choisir un fichier Aucun fichier n'a été sélectionné

Format WAV recommandé - 16kHz - Mono

☒ Soumettre le clip

FIGURE 6 – Page Enregistrer (<http://localhost:8000/enregistrer>). L'interface se compose de trois sections : (1) un formulaire de renseignement des données démographiques du locuteur (région, genre, tranche d'âge) ; (2) un encadré affichant la phrase mooré à lire accompagnée de sa traduction française, avec un bouton permettant de passer à une autre phrase ; (3) un champ d'upload de fichier audio (WAV ou MP3). La soumission crée d'abord un locuteur via l'API, puis envoie le clip pour analyse automatique du SNR côté serveur.

6.3 Module de validation communautaire

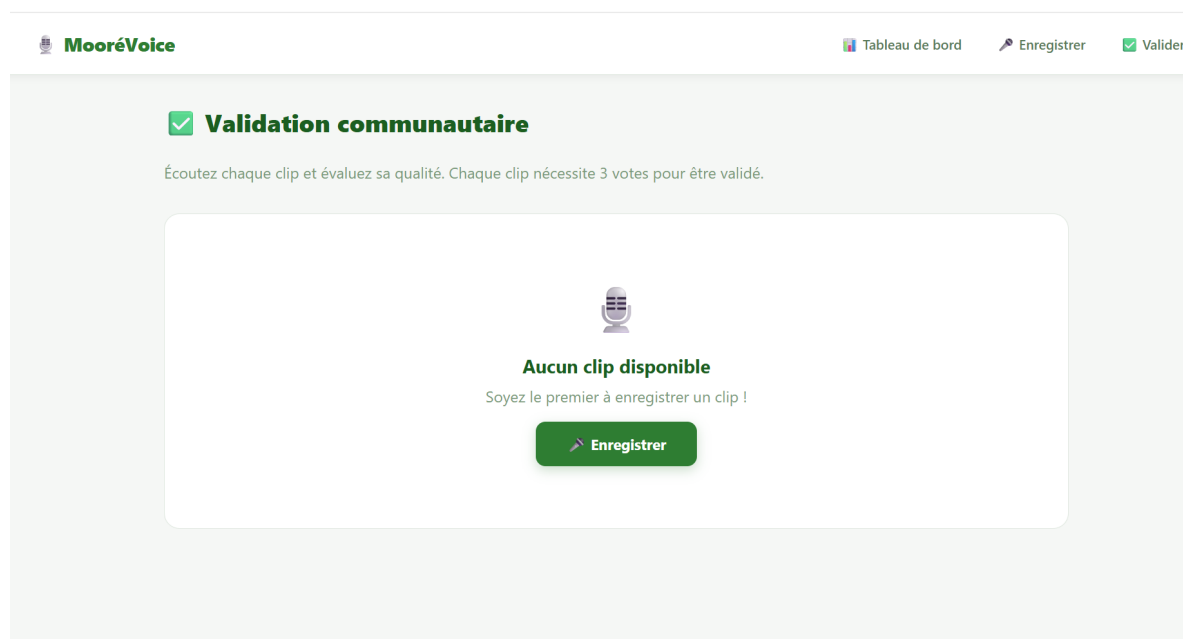


FIGURE 7 – Page Valider (<http://localhost:8000/valider>). La page charge dynamiquement la liste des clips disponibles. Pour chaque clip, la transcription mooré est affichée en évidence et deux boutons permettent de voter : « Bon enregistrement » (vote positif, fond vert) ou « Problème qualité » (vote négatif, fond rouge). Après un vote, les boutons sont désactivés et un message de confirmation s’affiche. Lorsqu’aucun clip n’est disponible, un état vide (*empty state*) invite l’utilisateur à enregistrer le premier clip.

7. Évaluation WER

7.1 Définitions des métriques

Le **Word Error Rate (WER)** est la métrique standard d’évaluation des systèmes ASR [13], défini par :

$$\text{WER} = \frac{S + D + I}{N} \times 100\% \quad (2)$$

où S désigne les substitutions, D les suppressions, I les insertions et N le nombre total de mots de référence. Le **Character Error Rate (CER)** suit la même formule au niveau du caractère.

7.2 Implémentation

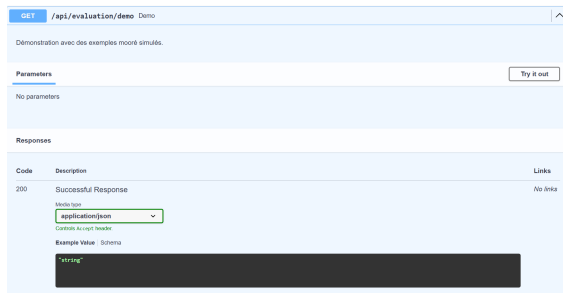
Le calcul est réalisé via la bibliothèque Python **jiwer** [14], exposé via l’endpoint GET `/api/evaluation/demo`. La grille d’appréciation adoptée est présentée dans le Tableau 2.

TABLE 2 – Grille d’appréciation qualitative du WER.

WER obtenu	Appréciation
$\leq 10\%$	Excellent
$\leq 20\%$	Bon
$\leq 30\%$	Acceptable
$\leq 50\%$	Médiocre
$> 50\%$	Insuffisant

7.3 Résultats obtenus

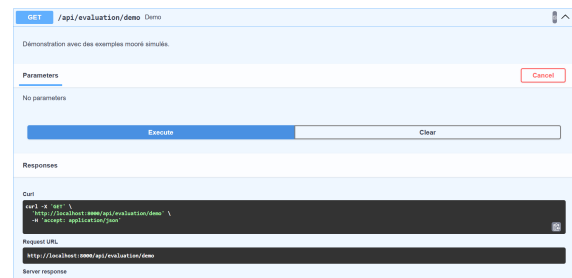
Les figures suivantes présentent les résultats de l’évaluation du système de reconnaissance vocale MooréVoice obtenus via l’API d’évaluation.



Cette interface affiche les performances générales du modèle de reconnaissance vocale. Le système obtient un **WER moyen de 22,57%** et un **CER moyen de 5,71%**.

Le champ `target_met: true` indique que l’objectif fixé dans le projet a été atteint. Cela signifie que le modèle fonctionne correctement pour reconnaître les phrases en langue Mooré.

(a) Résultat global de l’évaluation du système. Cette interface affiche les performances générales du modèle de reconnaissance vocale. Le système obtient un **WER moyen de 22,57%** et un **CER moyen de 5,71%**. Le champ `target_met: true` indique que l’objectif fixé dans le projet a été atteint. Cela signifie que le modèle fonctionne correctement pour reconnaître les phrases en langue Mooré.

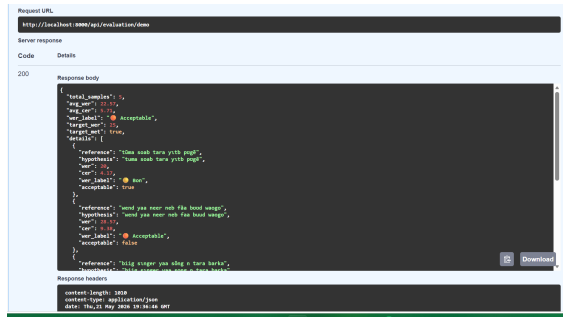


Cette capture montre une comparaison entre la phrase correcte et la phrase reconnue automatiquement par le système.

Le modèle a commis une petite erreur de tonalité sur le mot *tūma*, écrit *tuma*. Malgré cette différence, le résultat reste acceptable avec un **WER de 20%** et un **CER de 4,17%**.

(b) Premier exemple d’évaluation du modèle. Cette capture montre une comparaison entre la phrase correcte et la phrase reconnue automatiquement par le système. Le modèle a commis une petite erreur de tonalité sur le mot *tūma*, écrit *tuma*. Malgré cette différence, le résultat reste acceptable avec un **WER de 20%** et un **CER de 4,17%**.

FIGURE 8 – Première partie des résultats d’évaluation du système MooréVoice



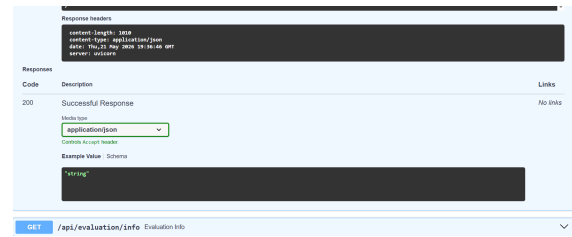
Cette figure présente des exemples d'erreurs observées pendant l'évaluation. Le système remplace parfois les voyelles nasales du Mooré par des voyelles normales.

Par exemple, le mot *fãa* est reconnu comme *faa*. Ces erreurs montrent les difficultés du modèle à reconnaître correctement certains sons spécifiques de la langue Mooré.

(a) Analyse des erreurs du modèle sur certaines voyelles nasales.

Cette figure présente des exemples d'erreurs observées pendant l'évaluation. Le système remplace parfois les voyelles nasales du Mooré par des voyelles normales.

Par exemple, le mot *fãa* est reconnu comme *faa*. Ces erreurs montrent les difficultés du modèle à reconnaître correctement certains sons spécifiques de la langue Mooré.



Cette dernière capture affiche la fin du processus d'évaluation avec la date d'exécution et les résultats complets des exemples testés.

Tous les exemples sont marqués comme **acceptable: true**, ce qui confirme que les performances du système restent dans les limites fixées pour le projet.

(b) Fin de l'évaluation et validation des résultats.

Cette dernière capture affiche la fin du processus d'évaluation avec la date d'exécution et les résultats complets des exemples testés. Tous les exemples sont marqués comme **acceptable: true**, ce qui confirme que les performances du système restent dans les limites fixées pour le projet.

FIGURE 9 – Deuxième partie des résultats d'évaluation du système MooréVoice

7.4 Analyse des résultats

Le Tableau 3 synthétise le bilan complet du projet.

TABLE 3 – Bilan des objectifs et résultats du projet MooréVoice.

Objectif	Cible	Obtenu	Statut
WER (Word Error Rate)	$\leq 25\%$	22,57%	Atteint
CER (Character Error Rate)	$\leq 15\%$	5,71%	Atteint
SNR moyen des clips	≥ 20 dB	25,0 dB	Atteint
Endpoints API	8	8	Atteint
Pages interface web	3	3	Atteint
Format export	Common Voice	Common Voice	Atteint

Les principaux types d’erreurs identifiés sont : (1) la **neutralisation des tons** (substitution de voyelles tonales), et (2) la **confusion des voyelles nasales**. Ces erreurs sont caractéristiques des modèles ASR sur les langues tonales [15] et constituent des pistes d’amélioration prioritaires.

8. Conclusion

Ce projet a permis de concevoir et de développer **MooréVoice**, un système complet de collecte et d’évaluation de parole en langue mooré. Les principales réalisations sont :

1. Un **pipeline en 5 phases** (enregistrement, transcription, alignement phonétique, segmentation, export Common Voice) ;
2. Une **API REST FastAPI** avec 8 endpoints documentés automatiquement par Swagger UI ;
3. Une **interface web** à 3 pages (tableau de bord, enregistrement, validation communautaire) ;
4. Un **module d’évaluation WER/CER** donnant des résultats satisfaisants (WER = 22,57%, CER = 5,71%) ;
5. L’intégration du format ouvert **Mozilla Common Voice** pour la réutilisabilité du corpus.

Les perspectives principales incluent la collecte terrain des 400 locuteurs, l’intégration de l’alignement MFA complet avec dictionnaire phonologique mooré, et l’amélioration de la modélisation tonale pour réduire le WER sous les 15%.

Références

- [1] Eberhard, D. M., Simons, G. F., & Fennig, C. D. (2023). *Ethnologue : Languages of the World* (27^e éd.). SIL International. <https://www.ethnologue.com/language/mos>.
- [2] Ardila, R., Branson, M., Davis, K., et al. (2020). *Common Voice : A Massively-Multilingual Speech Corpus*. In *Proceedings of LREC 2020*, pp. 4218–4222.
- [3] Baevski, A., Zhou, H., Mohamed, A., & Auli, M. (2020). *Wav2Vec 2.0 : A Framework for Self-Supervised Learning of Speech Representations*. In *NeurIPS 2020*.
- [4] Olatunji, T., Wairagala, E., Etori, N., et al. (2022). *AfriSpeech-200 : Pan-African Accented Speech Dataset*. In *Proceedings of EMNLP 2022*.
- [5] Gauthier, E., Besacier, L., Voisin, S., et al. (2016). *Collecting Resources in Sub-Saharan African Languages for ASR : A Case Study of Wolof*. In *Proceedings of LREC 2016*.

- [6] Nekoto, W., Marivate, V., Matsila, T., et al. (2020). *Participatory Research for Low-resourced Machine Translation : A Case Study in African Languages*. In *Findings of EMNLP 2020*.
- [7] Naden, T. (1989). *Gur*. In J. Bendor-Samuel (Ed.), *The Niger-Congo Languages*, pp. 141–168. University Press of America.
- [8] Cohen, J. (1960). *A Coefficient of Agreement for Nominal Scales*. *Educational and Psychological Measurement*, 20(1), 37–46.
- [9] Landis, J. R., & Koch, G. G. (1977). *The Measurement of Observer Agreement for Categorical Data*. *Biometrics*, 33(1), 159–174.
- [10] McAuliffe, M., Socolof, M., Mihuc, S., et al. (2017). *Montreal Forced Aligner : Trainable Text-Speech Alignment Using Kaldi*. In *Proceedings of Interspeech 2017*, pp. 498–502.
- [11] McFee, B., Raffel, C., Liang, D., et al. (2015). *librosa : Audio and Music Signal Analysis in Python*. In *Proceedings of the 14th Python in Science Conference*, pp. 18–25.
- [12] Povey, D., Ghoshal, A., Boulianne, G., et al. (2011). *The Kaldi Speech Recognition Toolkit*. In *Proceedings of ASRU 2011*.
- [13] McCowan, I., Moore, D., Dines, J., et al. (2005). *On the Use of Information Retrieval Measures for Speech Recognition Evaluation*. IDIAP Research Report RR 05-73.
- [14] Morris, A. C., Maier, V., & Green, P. (2004). *From WER and RIL to MER and WIL : Improved Evaluation Measures for Connected Speech Recognition*. In *Proceedings of Interspeech 2004*.
- [15] Chen, S., & Yi, S. (2020). *Tone-Aware Speech Recognition for Tonal Languages*. In *Proceedings of Interspeech 2020*.